Robots in Context

Lecture 2

Leon Bodenhagen lebo@mmmi.sdu.dk

Associate Professor

SDU Robotics, Maersk Mc-Kinney Moller Institute

Todays Lecture

Agenda

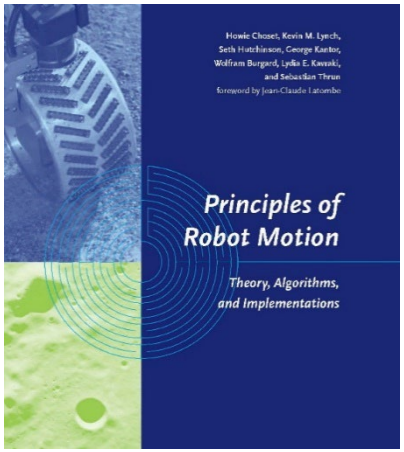
- ROS, lecture 1
- Why “Robots in Context”
- Bug Algorithms
- Programming Exercise

Practical info , book

Text book

- Shipped… (via UK)
- Available as pdf online too (free)
- Available on other platforms too

  (saxo, amazon, …)



Previous Lecture: ROS

Pollev.com/robots

Each Question will be one poll (upvote)

Q1 Did you succeed setting up ROS2

Q2 Which tutorials

Q3 Problems

Previous Lecture: ROS

Pollev.com/robots

Q1

How did you get ROS working?

How did you get ROS running? (Native Ubuntu, dual boot, windows, virtual machine ...)

Nobody has responded yet.

Hang tight! Responses are coming in.

Previous Lecture: ROS

Pollev.com/robots

Q2

What was the furthest tutorial / element you explored?

What was the most advanced tutorial you succeeded with?

Nobody has responded yet.

Hang tight! Responses are coming in.

Previous Lecture: ROS

Pollev.com/robots

Q3

What problems did you encounter?

What problems did you encounter? Did you solve it?

Nobody has responded yet.

Hang tight! Responses are coming in.

Progamming Exercise 1

At least 2 volunteers for PE1:

1.
2.

Deadline: 16/9

Todays Lecture

Agenda

- Why “Robots in Context”
- Bug Algorithms
- Programming Exercise

Robots in Context

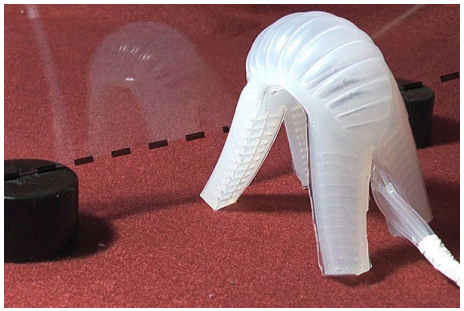
What is actually a robot?

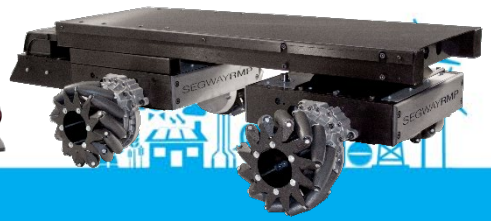
> *A robot is a machine—especially one programmable by a computer— capable of carrying out a complex series of actions automatically.*
>
> Oxford English Dictionary

Robots in Context

Robots can have very different properties

- Degrees of Freedom
- Configuration space
- Holonomic vs. non-holonomic





Robots in Context

What is actually context?

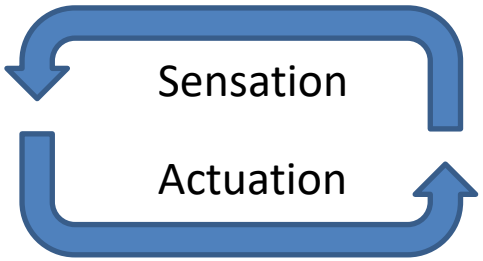
the set of circumstances or facts that surround a particular event, situation, etc.

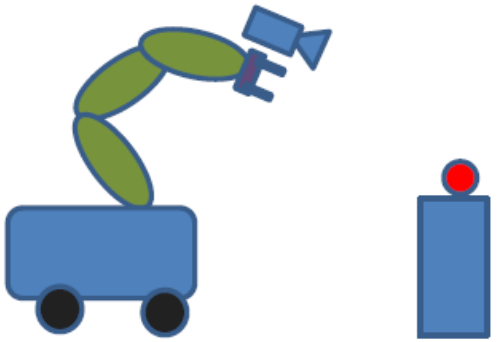
Dictionary.com

Robots in Context

What is actually context?

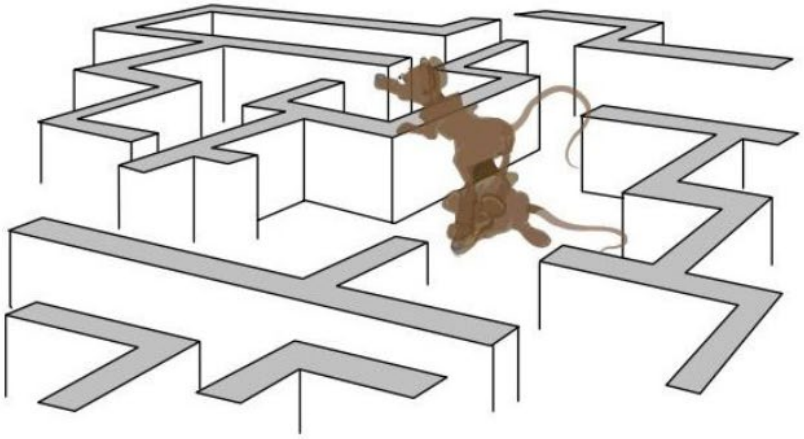
Robots should not be seen as stand-alone entities but operate in a context, meaning they must take this into account.





Motion Planning

- Determining where to go
  - More than a search...
  - Not only a robotics problem (animation/computer games, navigation, protein folding, ...)
  - Partial vs. Full information



Trends in Robotics / Motion Planning

Classical Robotics (mid-70's)

- exact models
- no sensing necessary

Reactive Paradigm (mid-80's)

- no models
- relies heavily on good sensing

Hybrids (since 90's)

- model-based at higher levels
- reactive at lower levels

Probabilistic Robotics (since mid-90's)

- seamless integration of models and sensing
- inaccurate models, inaccurate sensors

Trends in Robotics / Motion Planning

Classical Robotics (mid-70's)

- exact models
- no sensing necessary

Reactive Paradigm (mid-80's)

- no models
- relies heavily on good sensing

Hybrids (since 90's)

- model-based at higher levels
- reactive at lower levels

Probabilistic Robotics

- seamless integration of m
- inaccurate models, inaccu

Bug Algorithms

Simple Conceptual Motion Planning Algorithms

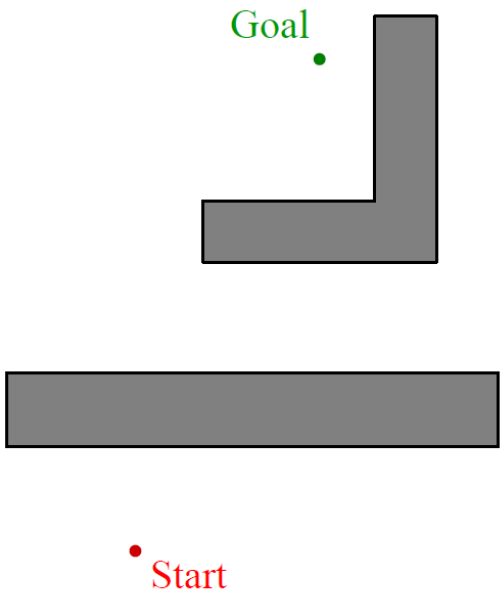
Bug Algorithms

- Many planning algorithms assume **global** knowledge
- Bug algorithms assume only **local** knowledge of the environment and a global goal
- Bug behaviors are simple:
  - Follow a wall (right or left)
  - Move in a straight line toward

A few general concepts

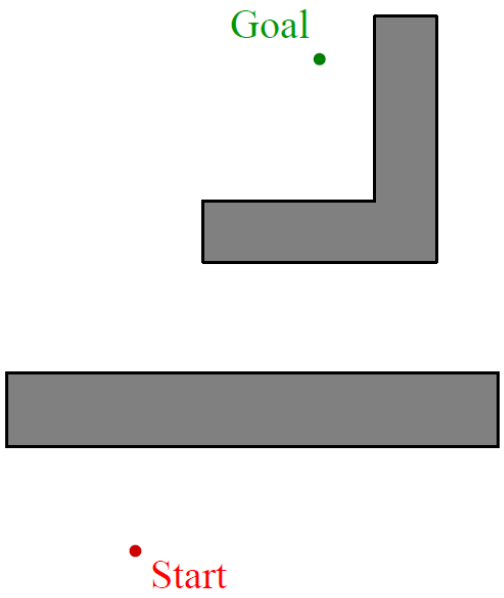
- For simple holonomic mobile robots the workspace W has the same dimensions as the configuration space C.
- WO_i denotes an obstacle

The Bug Algorithms



- Known direction and potentially distance to goal
- Only local sensing
  - walls/obstacles & encoders
- Reasonable world
  - Finitely many obstacles
  - a line will intersect an obstacle a finitely number of times
  - Workspace is bounded

Simplest strategy – Bug 0

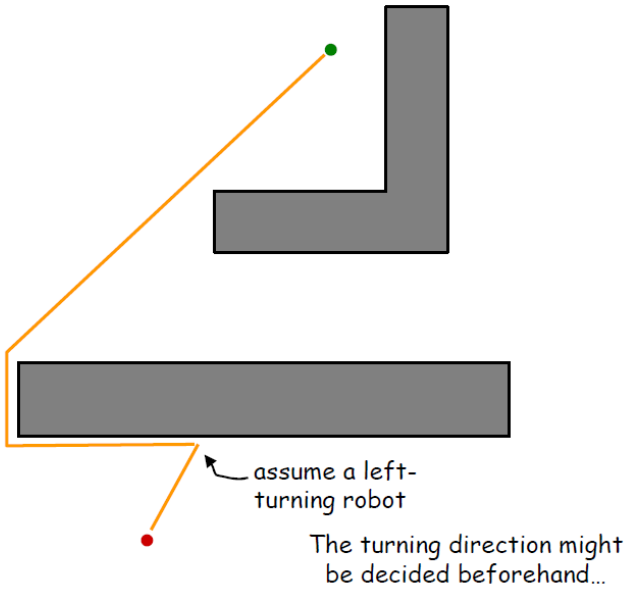


- Known direction to goal
- Only local sensing
  - walls/obstacles & encoders

Bug 0

1. Head towards goal
2. Follow obstacles until you can head toward goal again
3. Continue

Simplest strategy – Bug 0



- Known direction to goal
- Only local sensing
  - walls/obstacles & encoders

<u>Bug 0</u>

1. Head towards goal
2. Follow obstacles until you can head toward goal again
3. Continue

Bug Algorithms

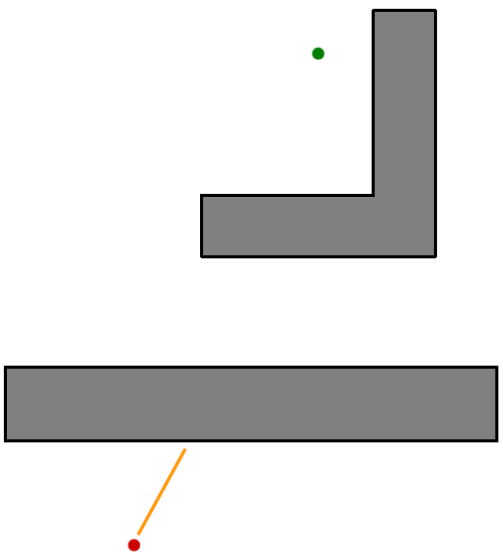
Can you come up with a map that makes Bug 0 fail?

- Known direction to goal
- Only local sensing
  - walls/obstacles & encoders

<u>Bug 0</u>

1. Head towards goal
2. Follow obstacles until you can head toward goal again
3. Continue

A better bug? Bug 1

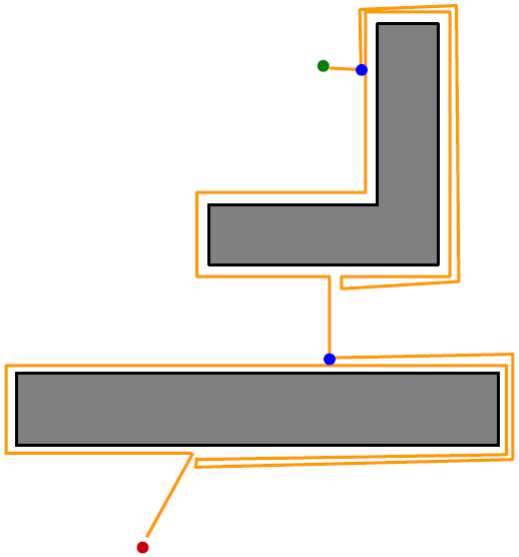


- Known direction and potentially distance to goal
- Only local sensing
  - walls/obstacles & encoders
- Add **memory**

Bug 1

1. Head towards goal
2. Circumnavigate and remember distance to goal
3. Return to point closest to goal and continue

A better bug? Bug 1

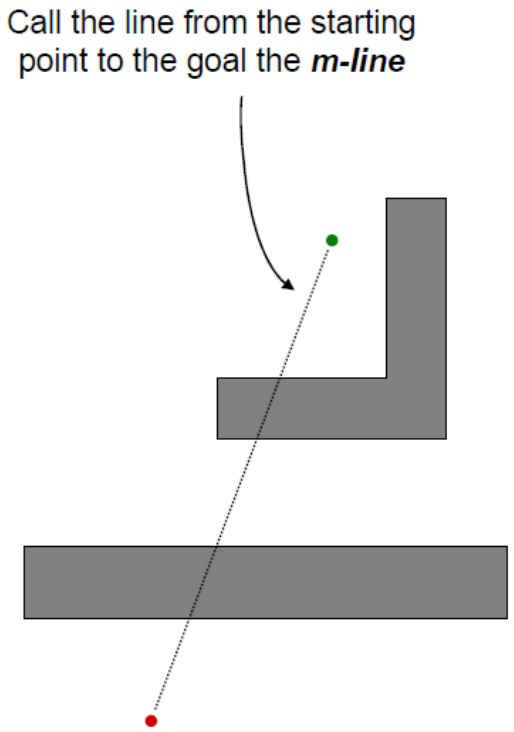


- Known direction and potentially distance to goal
- Only local sensing
  - walls/obstacles & encoders
- Add **memory**

Bug 1

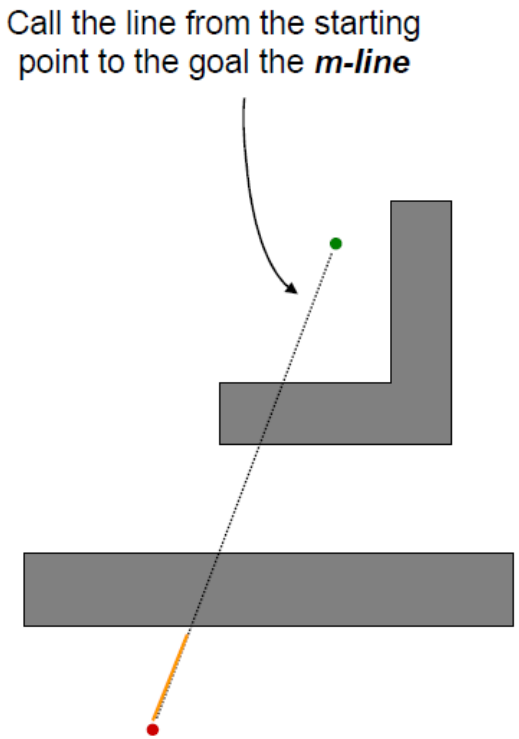
1. Head towards goal
2. Circumnavigate and remember distance to goal
3. Return to point closest to goal and continue

Bug 2



- Another attempt to improve the Bug algorithm

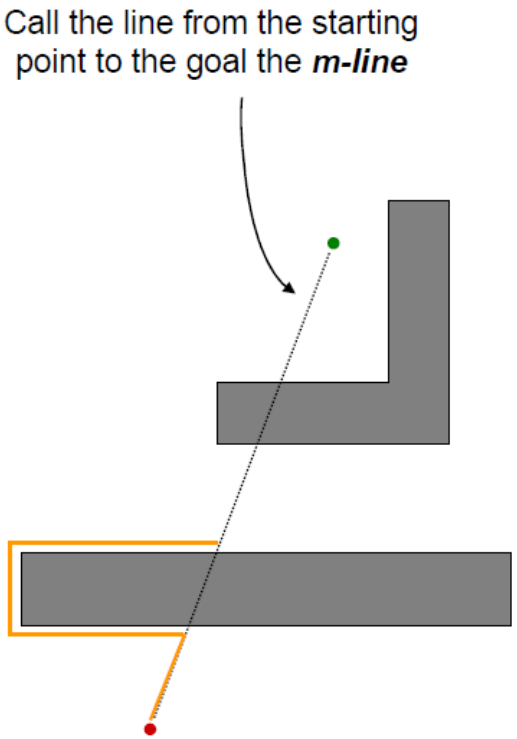
Bug 2



- Another attempt to improve the Bug algorithm

Bug 2
1. Head toward goal

Bug 2

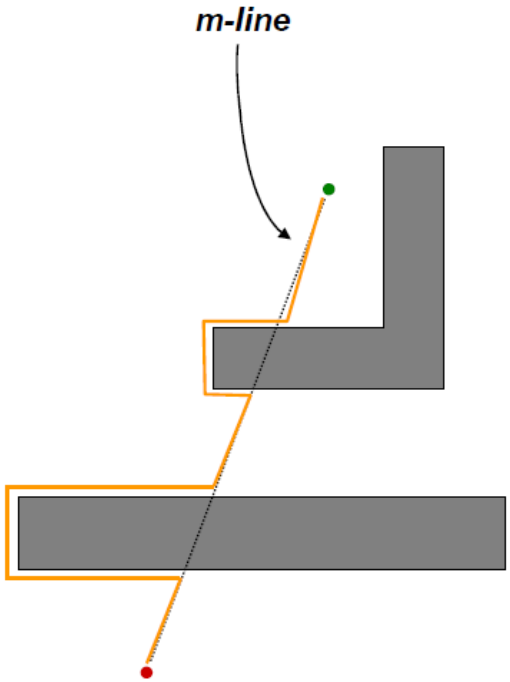


- Another attempt to improve the Bug algorithm

<u>Bug 2</u>

1. Head toward goal
2. Follow obstacle until hitting m-line again

Bug 2

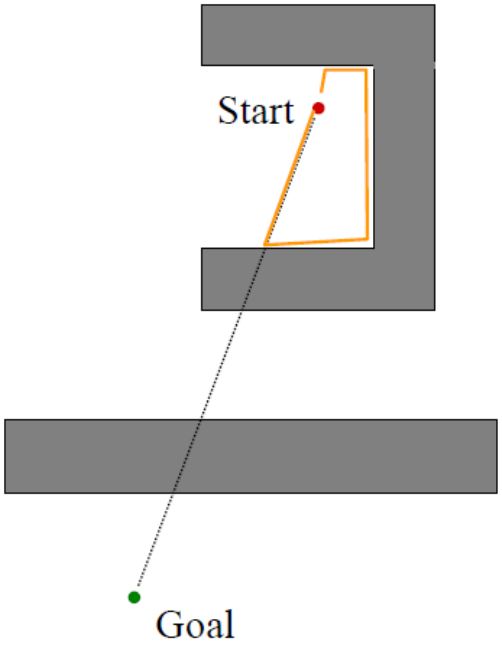


- Another attempt to improve the Bug algorithm

Bug 2

1. Head toward goal
2. Follow obstacle until hitting m-line again
3. Leave obstacle and continue toward goal

Bug 2 in trouble

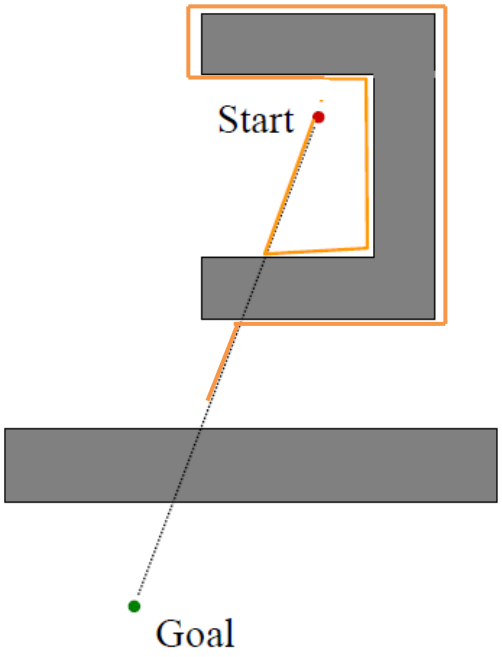


- Another attempt to improve the Bug algorithm

Bug 2

1. Head toward goal
2. Follow obstacle until hitting m-line again
3. Leave obstacle and continue toward goal

Bug 2



- Another attempt to improve the Bug algorithm

Bug 2

1. Head toward goal
2. Follow obstacle until hitting m-line again *closer to the goal*
3. Leave obstacle and continue toward goal

Bug Algorithms

- Bug 1 vs. Bug 2
  - Completeness: will they always find a solution if there is one?
  - What happens if they fail?
  - What is the worst case performance given a map with *n* obstacles, with a perimeter P_i?
    - Can you generate a simple map that where Bug 1 outperforms Bug 2?

A more realistic bug

Previous:

- Global beacons and contact based wall following

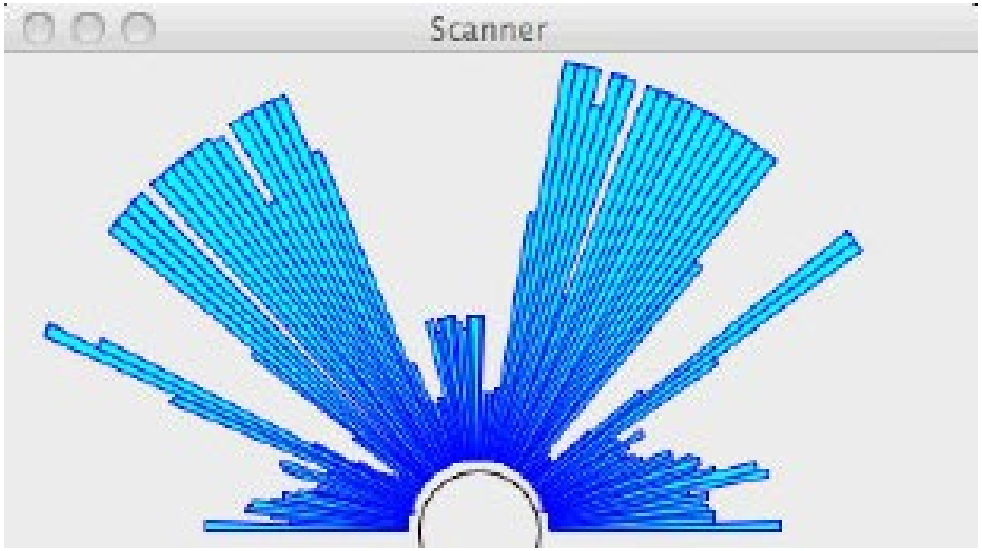
Reality:

- we typically use some sort of **range sensing** device that lets us look ahead (but has finite resolution and is noisy).

Let us assume we utilize a range sensor

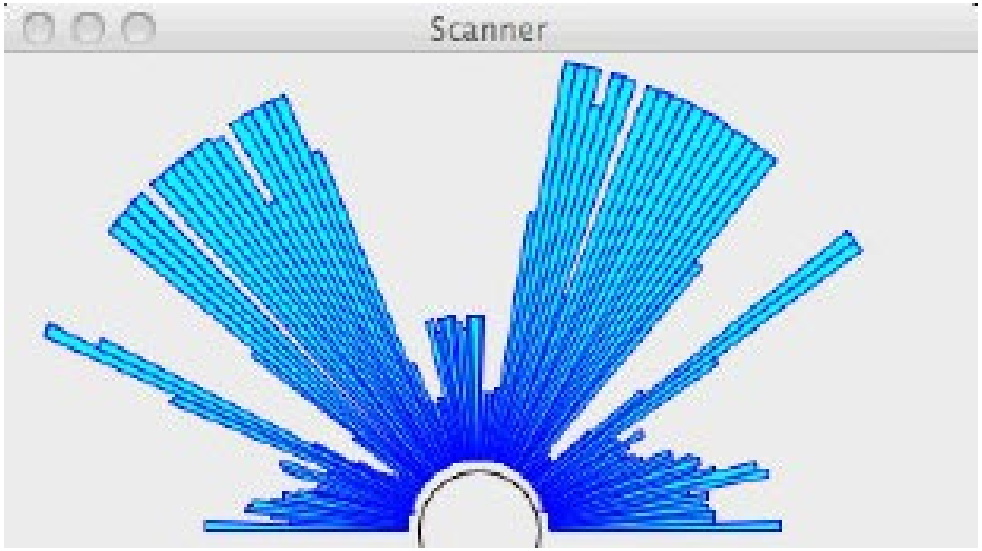
2D Range Sensor

- Output:
  - Angle
  - Distance
- Limited range



2D Range Sensor

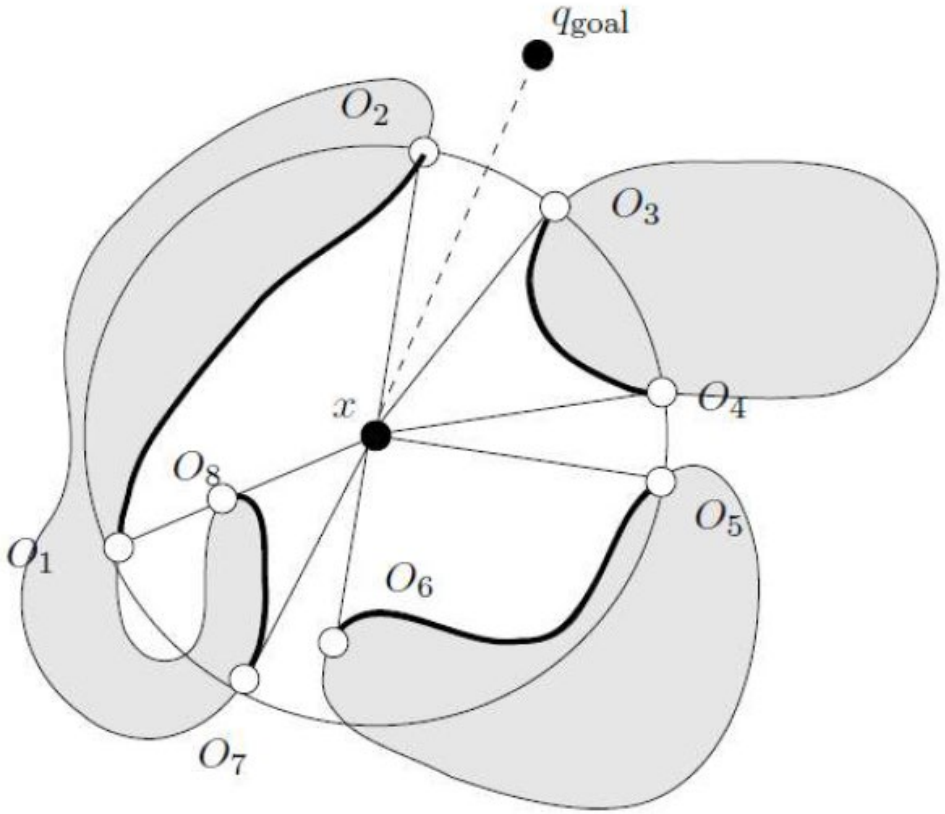
- Output:
  - Angle
  - Distance
- Limited range



$$\rho(x,\theta) = \min_{\lambda\in[0,\infty]} d(x, x+\lambda[\cos\theta, \sin\theta]^T),$$
$$\text{such that } x+\lambda[\cos\theta, \sin\theta]^T \in \bigcup_i \mathcal{WO}_i.$$

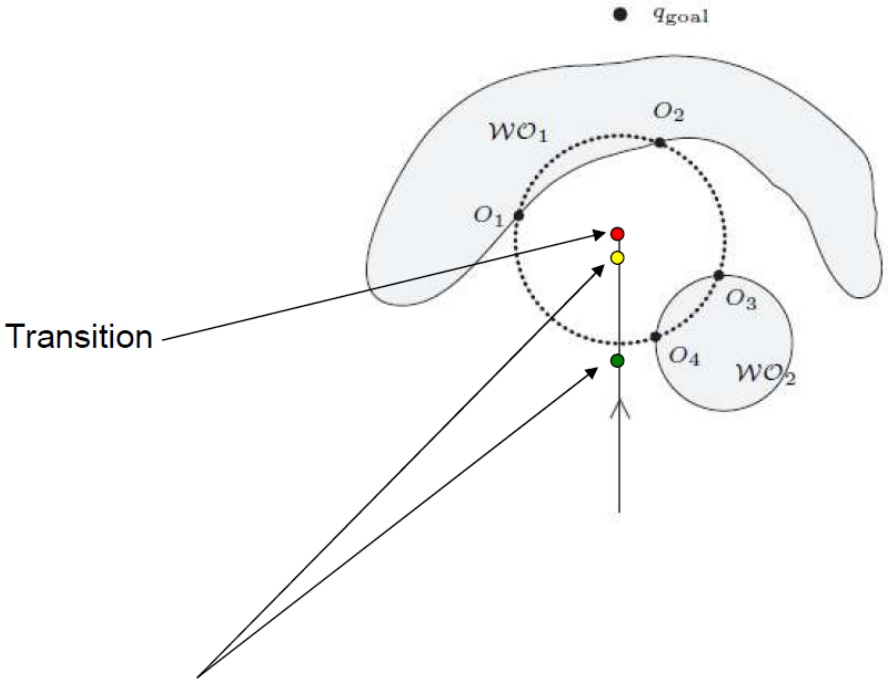
Intervals of continuity

- Tangent Bug relies on finding endpoints O_i of finite, continues segments



Tangent Bug: Moving-Toward-Goal

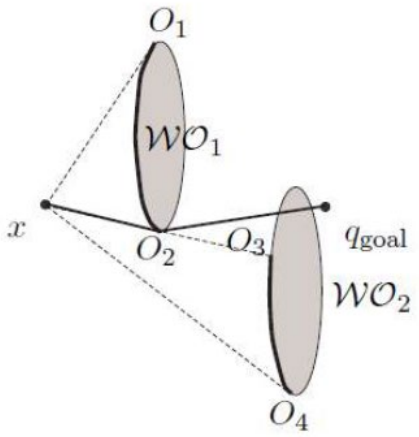
Moving-Toward-Goal (or discontinuity point O_i)



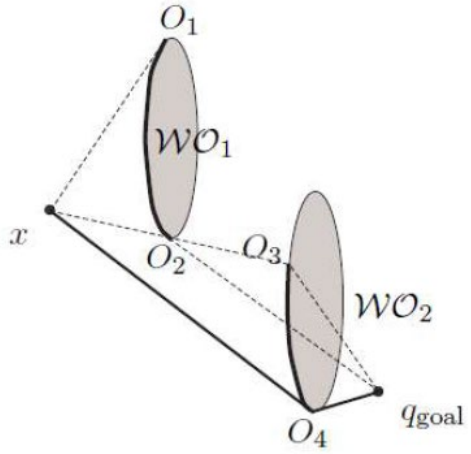
Idea: for any O_i such that $d(O_i, q_{goal}) < d(x, q_{goal})$
choose the O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$

Tangent Bug - Heuristics

At position x, the robot knows only what it sees and the position of the goal.



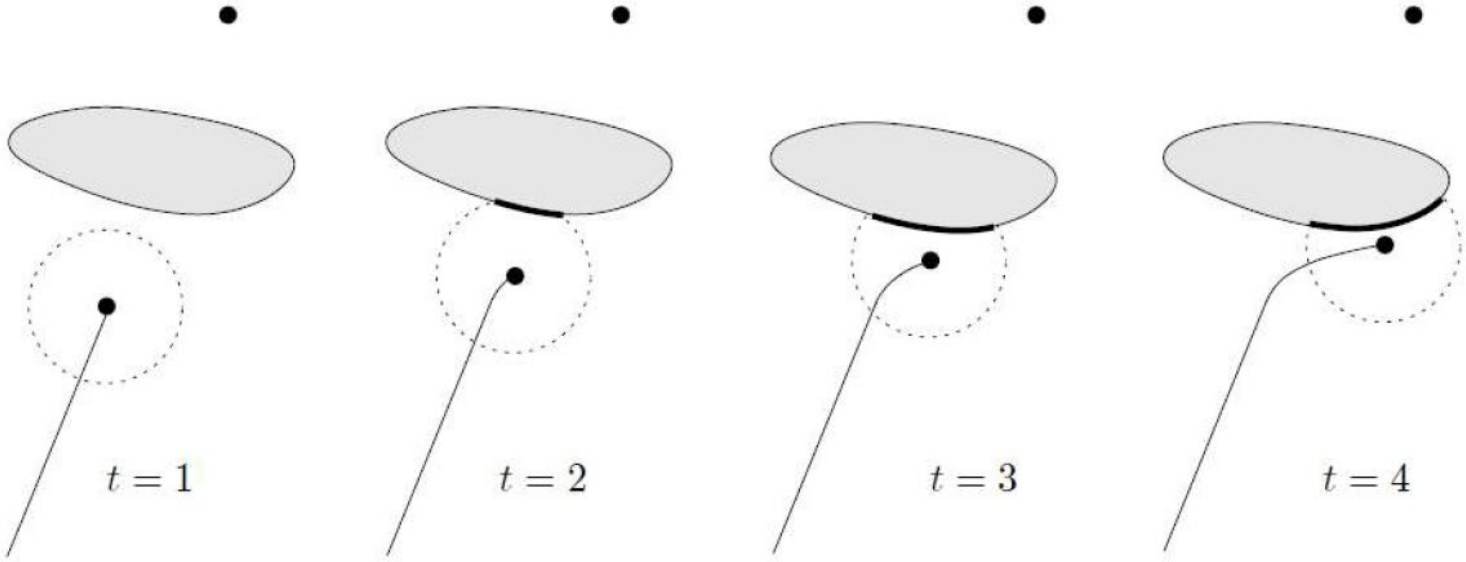
It moves toward O_2. It doesn't know yet that the line connecting O_2 and the goal intersects with an obstacle.



It moves toward O_4. It realizes that the line connecting O_2 and the goal intersects with an obstacle.

Idea: for any O_i such that $d(O_i, q_{goal}) < d(x, q_{goal})$
choose the O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$

Tangent Bug - Example



Idea: for any O_i such that $d(O_i, q_{goal}) < d(x, q_{goal})$
choose the O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$

Boundary Following

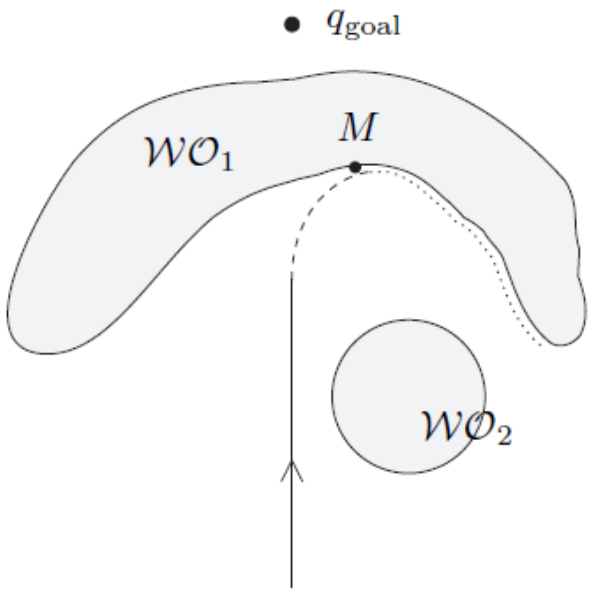
Idea: for any O_i such that $d(O_i, q_{goal}) < d(x, q_{goal})$
choose the O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$

Problem:

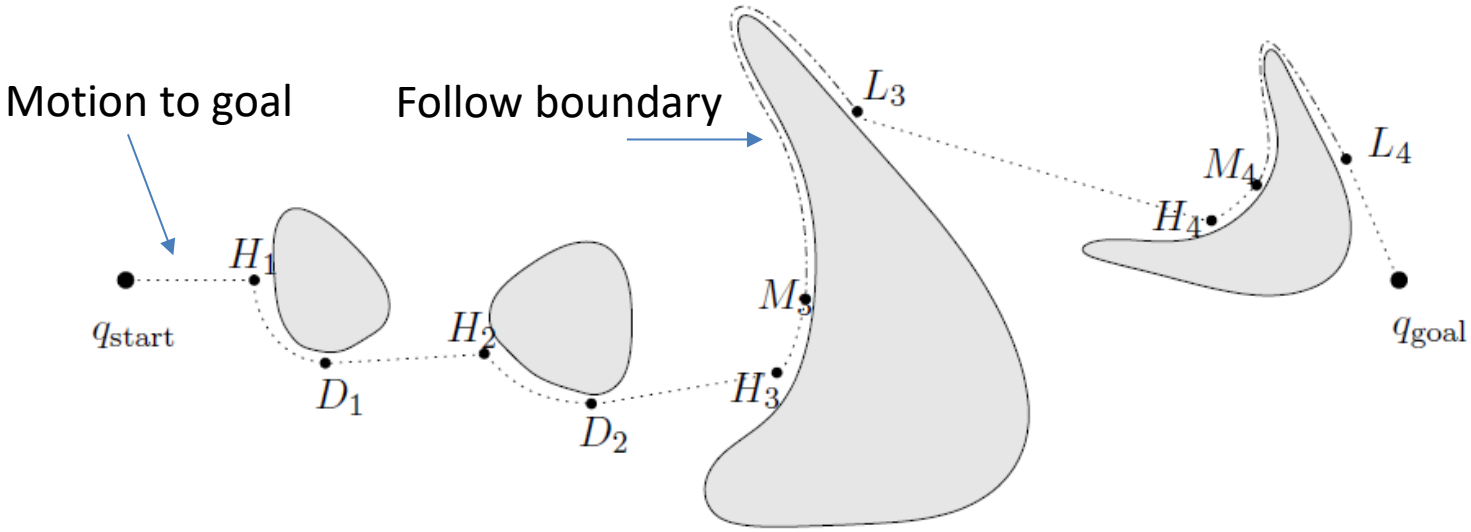
- What if the distance increases?

Solution:

- Follow the boundary
  - Closest distance to goal sensed previously: d_{Min}
  - Minimal departable distance to goal sensed now: d_{leave}
  - Stop following $d_{Min} > d_{leave}$

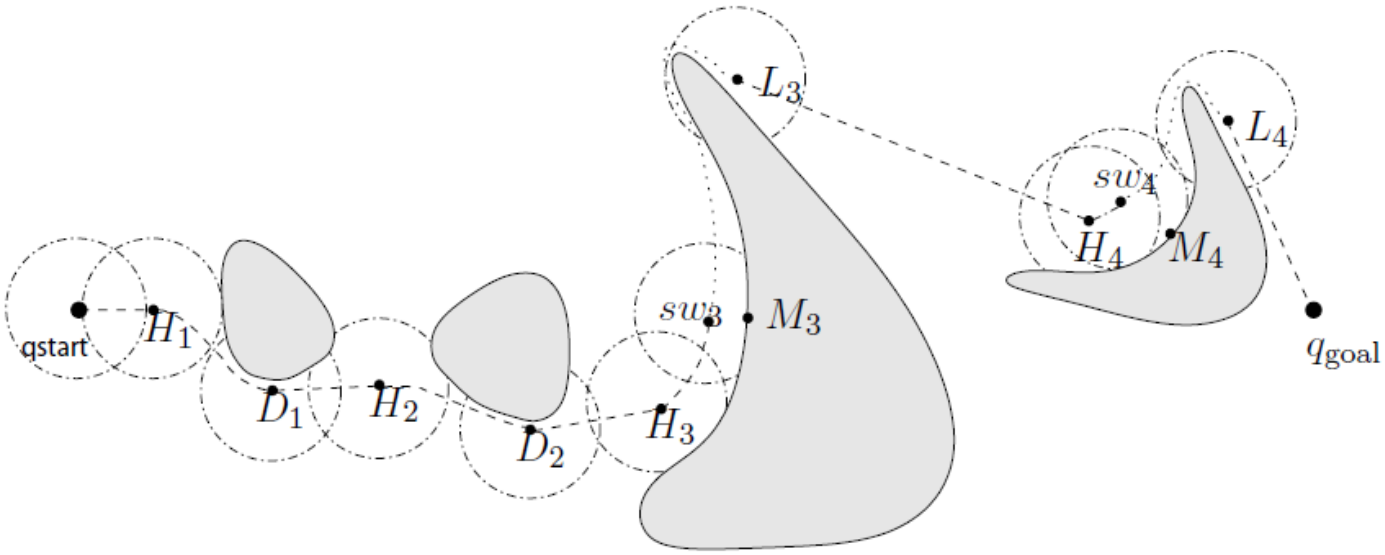


Example: zero range

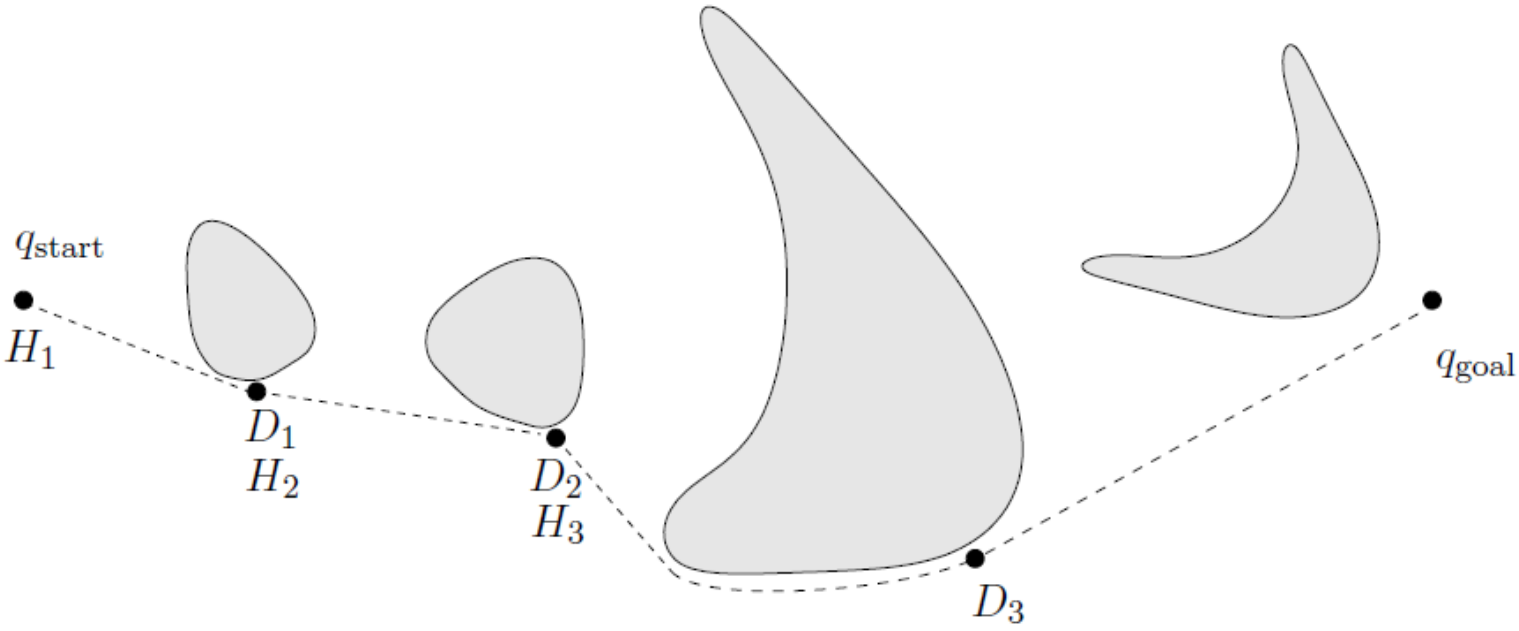


- Robot moves toward goal until it hits obstacle 1, WO_1, at hit point H_1
- Pretend there is an infinitely small sensor range and the O_i which minimizes the heuristic is to the right
- Keep following obstacle (move-to-goal) until robot can continue toward the goal, leaving object at depart point D_1
- Same situation with second obstacle
- At third obstacle, the robot turns left until it cannot decrease the heuristics further
- d_{min} denotes the distance between M_3 and the goal, d_{leave} equals the distance between robot and goal because sensing distance is zero. The boundary I followed until L_3

Example: finite range



Example: infinite range



Basic ideas - summary

- A **motion-to-goal** behavior as long as way is clear *or* there is a visible obstacle boundary point that **decreases the heuristic distance**
- A **boundary following** behavior invoked when **heuristic distance increases**.
  - A value d_{min} which is the shortest distance observed thus far between the sensed boundary of the obstacle and the goal
  - A value d_{leave} which is the shortest distance between any point in the currently sensed environment and the goal
  - Terminate boundary following behavior when $d_{leave} < d_{min}$

Bug algorithms, summary

- There are quite a few bugs to choose among
- Different options
  - Completeness vs. efficiency
  - Sensors required
- All composed by:
  - Moving towards the goal
  - Circumventing obstacles

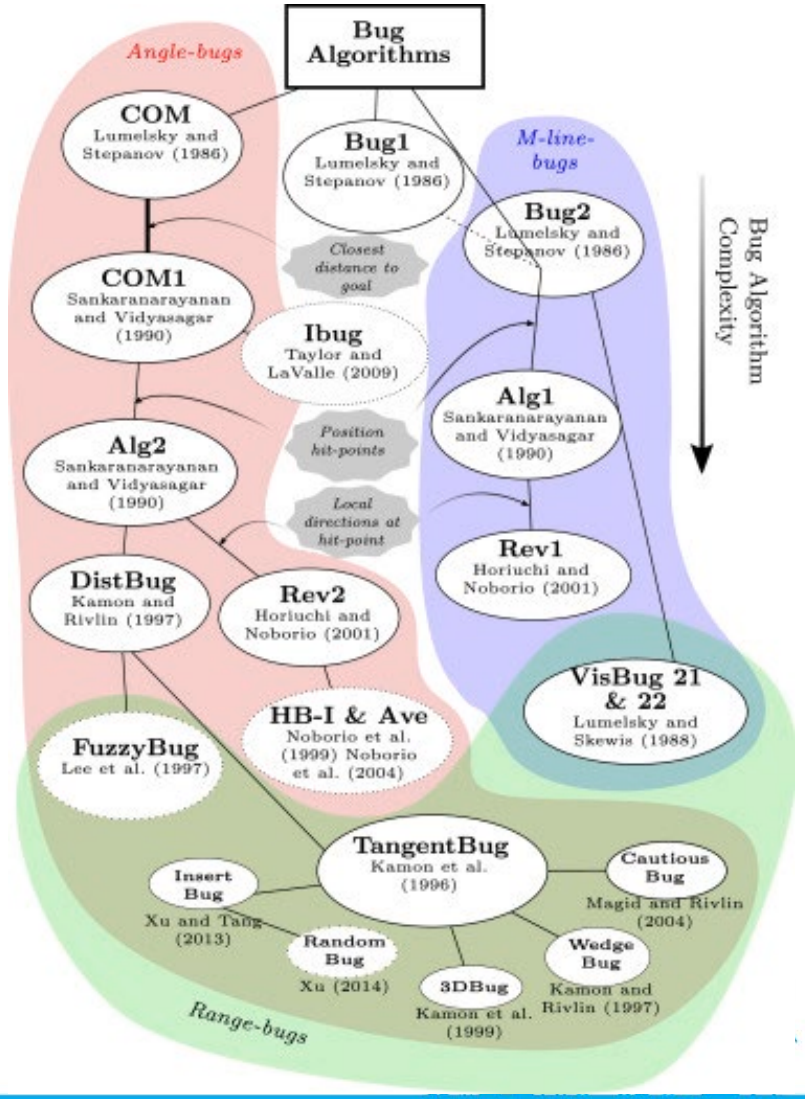


Figure credit: A comparative study of bug algorithms for robot navigation;McGuire et al.,RAS,2019

Programming Exercise

- Implement ideally two bug algorithms and compare them using the map on blackboard
- See details in the weekly note and separate presentation.